



# Airoha IoT SDK for BT Data Build Environment Guide

Version: 2.0  
Release date: 19 Mar 2026

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

## Airoha IoT SDK for BT Data Build Environment Guide

### Document Revision History

| Revision | Date        | Description                                                                                                                           |
|----------|-------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1.0      | 26 Sep 2025 | <ul style="list-style-type: none"><li>Initial release.</li><li>Added AB162x build environment for the BT Data product line.</li></ul> |
| 2.0      | 19 Mar 2026 | <ul style="list-style-type: none"><li>Added AB161x information.</li></ul>                                                             |

# Airoha IoT SDK for BT Data

## Build Environment Guide

### Table of Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Document Revision History.....</b>                      | <b>2</b>  |
| <b>Table of Contents.....</b>                              | <b>3</b>  |
| <b>List of Figures.....</b>                                | <b>3</b>  |
| <b>List of Tables .....</b>                                | <b>4</b>  |
| <b>1 Overview .....</b>                                    | <b>5</b>  |
| <b>2 Build Environment.....</b>                            | <b>6</b>  |
| 2.1 Docker Environment .....                               | 6         |
| 2.2 Docker Image .....                                     | 9         |
| 2.3 User Build Environment .....                           | 9         |
| <b>3 Folder Structure.....</b>                             | <b>12</b> |
| <b>4 Building a Project using the SDK .....</b>            | <b>13</b> |
| <b>5 Creating and Configuring a Project .....</b>          | <b>15</b> |
| 5.1 Creating a New Project Using an Existing Project ..... | 15        |
| 5.2 Configuring a Project.....                             | 16        |
| <b>Exhibit 1 Terms and Conditions.....</b>                 | <b>21</b> |

### List of Figures

|                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------|----|
| Figure 1. Architecture of Airoha IoT SDK Build Environment for BT Data Products .....                        | 6  |
| Figure 2. Successful Installation of WSL2 .....                                                              | 7  |
| Figure 3. Ubuntu 22.04.5 LTS in Microsoft Store .....                                                        | 8  |
| Figure 4. Executing Commands on Ubuntu 22.04.5 LTS .....                                                     | 8  |
| Figure 5. Contents of the Airoha IoT SDK Package.....                                                        | 9  |
| Figure 6. Screenshot after Executing the Above Commands.....                                                 | 10 |
| Figure 7. Screenshot of Building the Multi_Thread Sample Project .....                                       | 13 |
| Figure 8. Project Setting of Multi_Thread for Board ab1623m_evk .....                                        | 13 |
| Figure 9. Build Result of Multi_Thread for Board ab1623m_evk.....                                            | 14 |
| Figure 10. Screenshot after Copying Multi_Thread to Example_Project.....                                     | 15 |
| Figure 11. Screenshot of Board Configuration in Example_Project .....                                        | 15 |
| Figure 12. Screenshot of Board Configuration in Example_Project after Deleting Unnecessary Files .....       | 15 |
| Figure 13. Project Settings Related to Example_Project .....                                                 | 16 |
| Figure 14. Overall Flow of Configuring Kconfig Options of a Project .....                                    | 17 |
| Figure 15. Screenshot of the Results after Executing Menuconfig.py .....                                     | 18 |
| Figure 16. Screenshot of the Results after Entering '/', Entering 'daemon' and Pressing the Down Arrow ..... | 18 |
| Figure 17. Kconfig Option Adjustment Window .....                                                            | 19 |
| Figure 18. The Result Files after Changing the Kconfig Options .....                                         | 19 |
| Figure 19. Difference between .config and .config.old .....                                                  | 20 |
| Figure 20. Merge Temporarily Changes to Configuration File.....                                              | 20 |

Airoha IoT SDK for BT Data  
Build Environment Guide

List of Tables

Table 1. Introduction to Parameters in Docker Run Command ..... 10

Table 2. Introduction of Important Folders in the Airoha IoT SDK..... 12

Table 3. Syntax and Examples of Using Build.py ..... 14

## Airoha IoT SDK for BT Data Build Environment Guide

# 1 Overview

This guide provides information about the tools and utilities for setting up the build environment and running projects with the Airoha Internet of Things (IoT) Software Development Kit (SDK).

It provides information about the subsequent items:

- Setting up the build environment
- Folder structure of the SDK
- Building a project using the SDK
- Adding a module to the subsys
- Creating and configuring a project

This information can be applied to the Airoha IoT Development Platform, including the AB1615, AB1615M, AB1615DM, AB1616M, AB1620D, AB1623, AB1620DM, AB1623M, and AB1625M evaluation kits (EVK).

## Airoha IoT SDK for BT Data Build Environment Guide

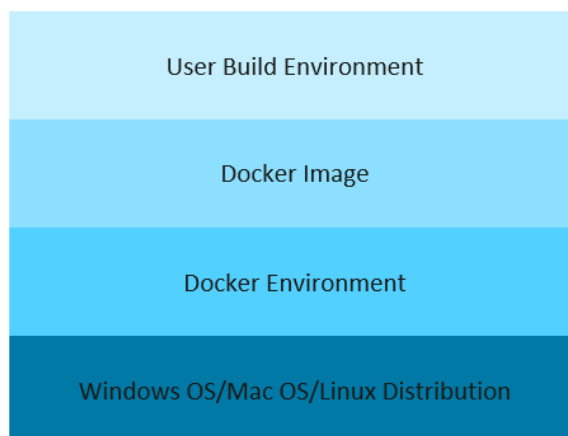
## 2 Build Environment

This section provides detailed information on setting up the SDK build environment on commonly used Linux distributions, macOS, and Microsoft Windows operating systems.

Airoha IoT SDK is based on Zephyr OS, which is a small-footprint kernel designed for use on resource-constrained and embedded systems.

The Zephyr Project releases an integrated environment suitable for Zephyr project development in the GitHub repository [zephyrproject-rtos/docker-image](https://github.com/zephyrproject-rtos/docker-image). Developers can use it to deploy the development environment with one click and quickly start project development.

Airoha downloads the Docker image of Zephyr from the network and packages it within the SDK to facilitate rapid product development for customers.



**Figure 1. Architecture of Airoha IoT SDK Build Environment for BT Data Products**

### 2.1 Docker Environment

To ensure consistency in project build processes and performance, Airoha recommends using the Docker environment.

#### 2.1.1 Setting Up Docker Environment on Windows

For Windows users to run Docker images, complete the following steps:

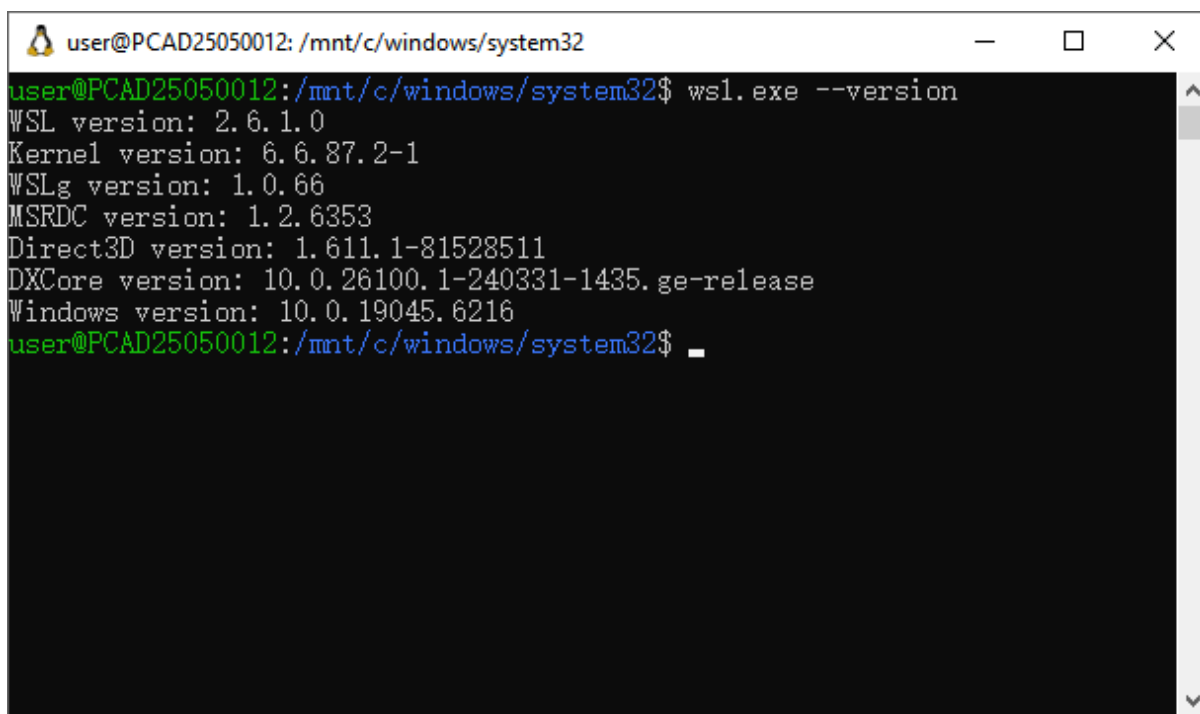
- Install WSL2.
- Install Ubuntu 22.04.5 LTS.
- Install Docker and other fundamental software.

The following explains the installation steps mentioned above.

First, to run a Linux distribution on Windows, refer to [How to install Linux on Windows with WSL](#) to complete the installation of WLS2 (Windows Subsystem for Linux version 2). Issues encountered during the installation and usage of WSL2 can be resolved by referring to [Troubleshooting Windows Subsystem for Linux](#).

## Airoha IoT SDK for BT Data Build Environment Guide

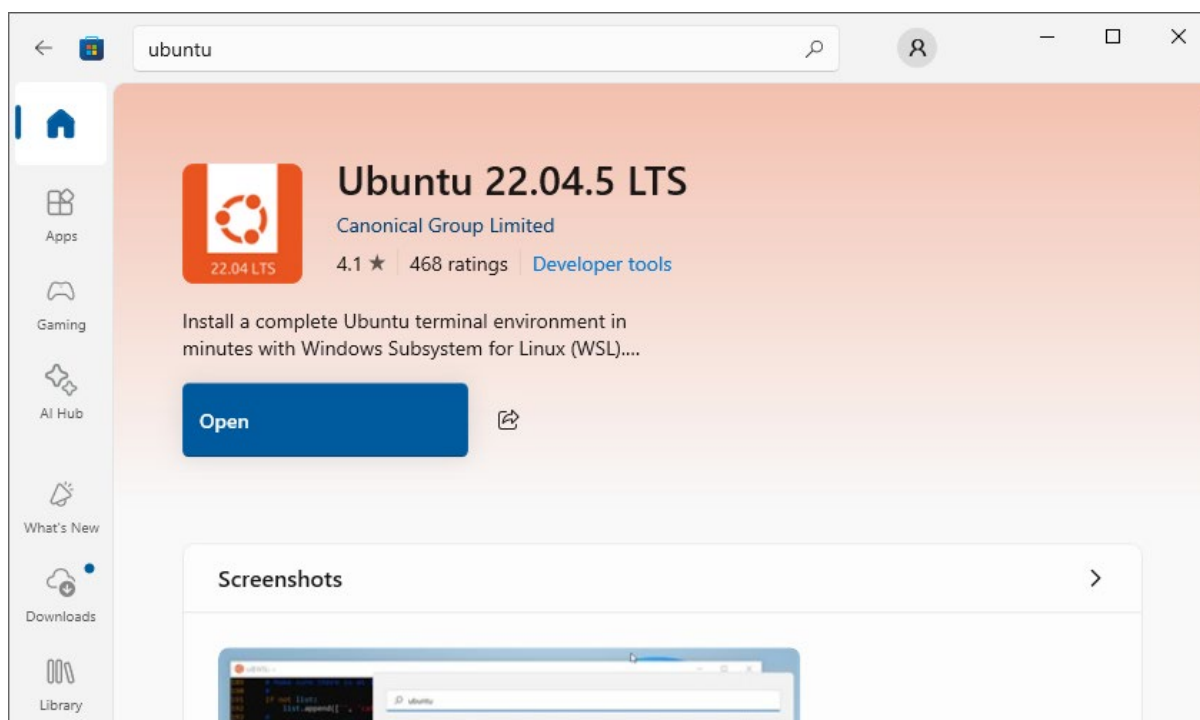
Figure 2 shows the successful installation and execution of WSL2. The screenshot indicates that WSL2 has been correctly installed, as its version number begins with “2”. Relevant version information is also shown but will not be elaborated here.



```
user@PCAD25050012: /mnt/c/windows/system32
user@PCAD25050012:/mnt/c/windows/system32$ wsl.exe --version
WSL version: 2.6.1.0
Kernel version: 6.6.87.2-1
WSLg version: 1.0.66
MSRDC version: 1.2.6353
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.19045.6216
user@PCAD25050012:/mnt/c/windows/system32$
```

**Figure 2. Successful Installation of WSL2**

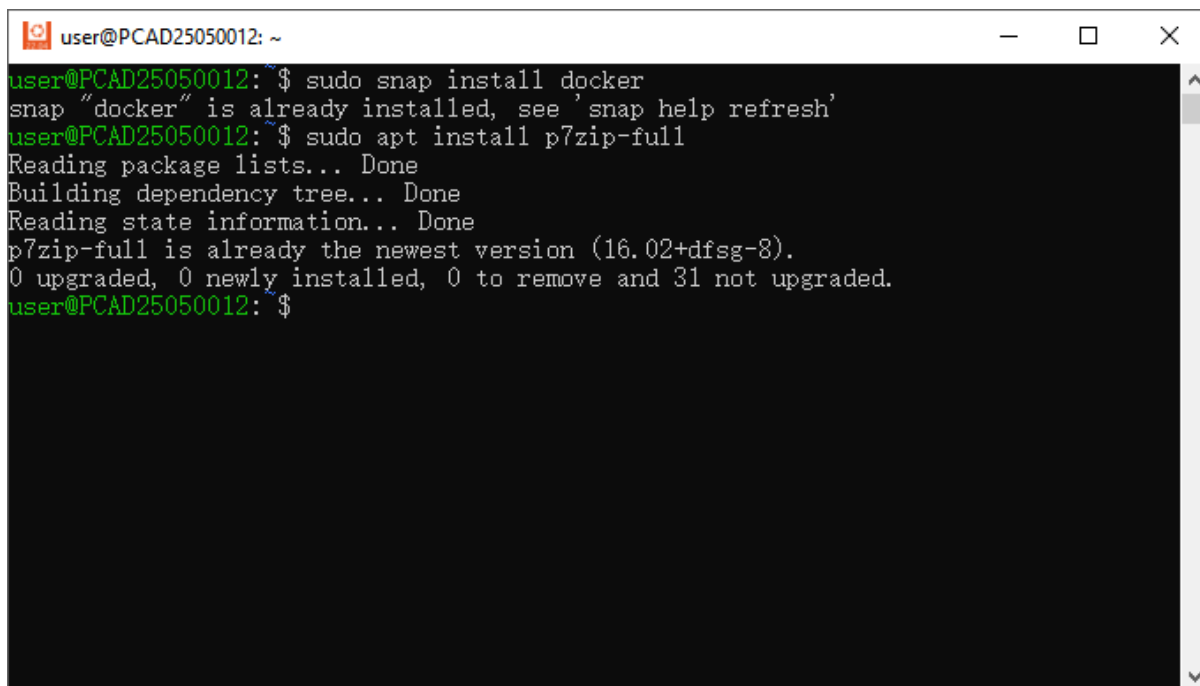
In the second step, open the Microsoft Store, search for Ubuntu 22.04.5 LTS and install it. The following figure shows a screenshot after the installation.



## Airoha IoT SDK for BT Data Build Environment Guide

*Figure 3. Ubuntu 22.04.5 LTS in Microsoft Store*

Finally, open the installed Ubuntu WSL system and install the Docker environment and some basic software.



```

user@PCAD25050012: ~
user@PCAD25050012:~$ sudo snap install docker
snap "docker" is already installed, see 'snap help refresh'
user@PCAD25050012:~$ sudo apt install p7zip-full
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
p7zip-full is already the newest version (16.02+dfsg-8).
0 upgraded, 0 newly installed, 0 to remove and 31 not upgraded.
user@PCAD25050012:~$

```

*Figure 4. Executing Commands on Ubuntu 22.04.5 LTS*

As shown in Figure 4, after installing the Ubuntu WSL2 system, click "**Open**" and follow the instructions to set up the user account and password. Then, enter the installation command in the command-line interface to install the Docker environment and the required basic software.

```

/* installation commands of docker environment and essential software */
/* Install the Docker environment to provide basic functionality for
loading and running Docker images. */
$sudo snap install docker

/* The Airoha IoT SDK is compressed using the 7z algorithm, providing a
high compression ratio. It needs to be decompressed using the 7z command.
*/
$sudo apt install p7zip-full

```

### 2.1.2 Setting Up Docker Environment on Ubuntu

Refer to the last step of Section 2.1.1 Setting Up Docker Environment on Windows to complete the installation of the Docker environment and basic software in Ubuntu.

### 2.1.3 Setting Up Docker Environment on MacOS

Refer to the [Install Docker Desktop in Mac](#) for detailed installation steps.

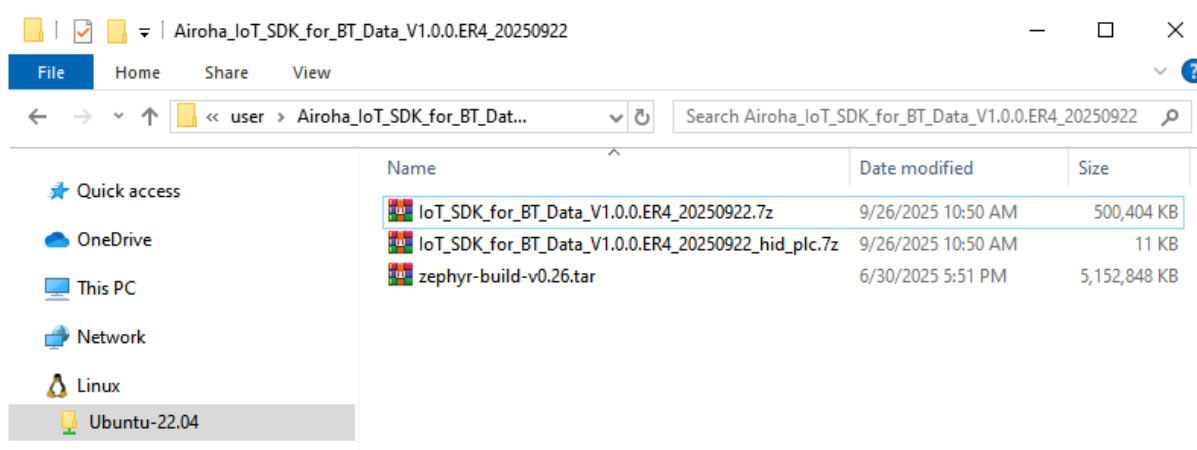
## Airoha IoT SDK for BT Data Build Environment Guide

### 2.2 Docker Image

A Docker image is a lightweight, standalone, executable software package that contains everything needed to run an application, including code, runtime, libraries, environment variables, and configuration files.

During the development process based on Zephyr OS, developers can obtain various environments required for Zephyr development from [docker-image](#), including but not limited to Python, CMake, Devicetree compiler, and various Python libraries that Zephyr depends on.

To further facilitate developers, Airoha has adapted the Zephyr Docker image for the SDK, downloading and packaging it within the SDK package. The following figure illustrates the contents included in the SDK package.



**Figure 5. Contents of the Airoha IoT SDK Package**

The Airoha IoT SDK includes the following file types:

- The code repository file, which includes the version number and date information.
- The add-on code repository, which includes version and date information.
- The Docker image that Airoha packaged from the [docker-image](#) to facilitate SDK development.

### 2.3 User Build Environment

After establishing the Docker runtime environment, Docker image, and SDK, execute the following commands to quickly deploy the developer environment.

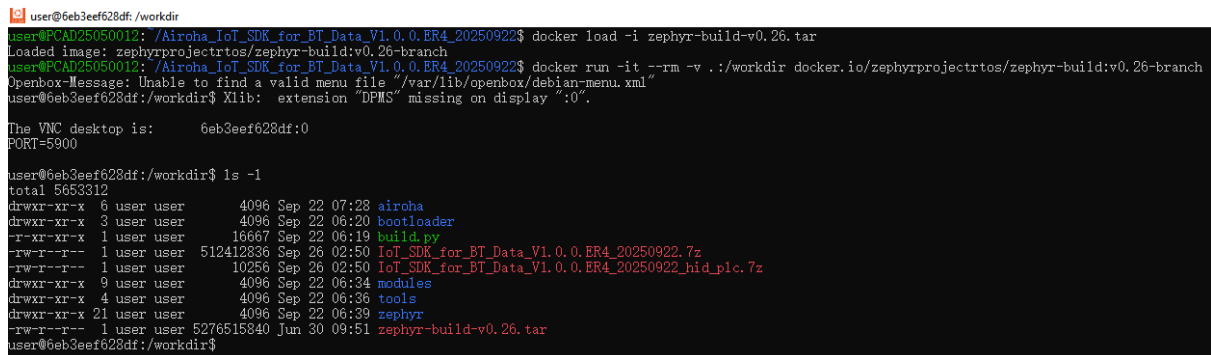
## Airoha IoT SDK for BT Data Build Environment Guide

```
/* Unzip the SDK's main code base in the current directory. */
$7z x IoT_SDK_for_BT_Data_V1.0.0.ER4_20250922.7z

/* Unzip the add-on code that the developer's company requested from Airoha
in the current directory. */
$7z x IoT_SDK_for_BT_Data_V1.0.0.ER4_20250922_hid_plc.7z

/* Load the docker image with the zephyr development environment. */
$docker load -i zephyr-build-v0.26.tar

/* Execute the docker image with the zephyr development environment. */
$docker run -it --rm -v ./workdir docker.io/zephyrprojectrtos/zephyr-
build:v0.26-branch
```



```
user@6eb3eef628df: /workdir
user@6eb3eef628df: /workdir$ docker load -i zephyr-build-v0.26.tar
Loaded image: zephyrprojectrtos/zephyr-build:v0.26-branch
user@6eb3eef628df: /workdir$ docker run -it --rm -v ./workdir docker.io/zephyrprojectrtos/zephyr-build:v0.26-branch
Openbox-Message: Unable to find a valid menu file "/var/lib/openbox/debian-menu.xml"
user@6eb3eef628df: /workdir$ Xlib: extension "DPMS" missing on display ":0".

The VNC desktop is:      6eb3eef628df:0
PORT=5900

user@6eb3eef628df: /workdir$ ls -l
total 5653312
drwxr-xr-x  6 user user    4096 Sep 22 07:28 airoha
drwxr-xr-x  3 user user    4096 Sep 22 06:20 bootloader
-r-xr-xr-x  1 user user    16667 Sep 22 06:19 build.py
-rw-r--r--  1 user user 512412836 Sep 26 02:50 IoT_SDK_for_BT_Data_V1.0.0.ER4_20250922.7z
-rw-r--r--  1 user user 10256 Sep 26 02:50 IoT_SDK_for_BT_Data_V1.0.0.ER4_20250922_hid_plc.7z
drwxr-xr-x  9 user user    4096 Sep 22 06:34 modules
drwxr-xr-x  4 user user    4096 Sep 22 06:36 tools
drwxr-xr-x 21 user user    4096 Sep 22 06:39 zephyr
-rw-r--r--  1 user user 5276515840 Jun 30 09:51 zephyr-build-v0.26.tar
user@6eb3eef628df: /workdir$
```

**Figure 6. Screenshot after Executing the Above Commands**

Note: The Docker load command does not display a progress bar. Depending on the performance of the developer's computer, it may take 3 to 15 minutes.

Additionally, after executing the Docker run command, if the window displays **"PORT=5900"**, press the **Enter** key to access the command line window and avoid unnecessary waiting.

In the above commands, the initial commands are relatively simple and will not be elaborated here. The Docker run command is slightly more complex and will be briefly introduced in Table 1.

**Table 1. Introduction to Parameters in Docker Run Command**

| Parameter | Meaning                                                                                                                                                                                                 |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| --it      | These two options are used together. The -l option means running the container in interactive mode, and -t allocates a pseudo terminal. This allows performing interactive operations in the container. |
| --rm      | This option means that the container will be automatically deleted after it exits. This prevents the container from continuing to occupy system resources after exiting.                                |

## Airoha IoT SDK for BT Data Build Environment Guide

| Parameter                                             | Meaning                                                                                                                                                                                                                                                         |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ./workdir                                             | It mounts the current directory (.) to the /workdir directory inside the container. This makes the files in the current directory accessible and modifiable inside the container.                                                                               |
| docker.io/zephyrprojectrtos/zephyr-build:v0.26-branch | This is the name and tag of the Docker image you want to run. docker.io is the default Docker image registry, zephyrprojectrtos/zephyr-build is the name of the image, and v0.26-branch is the tag of the image, which represents a specific version or branch. |

## Airoha IoT SDK for BT Data Build Environment Guide

### 3 Folder Structure

The following table displays some important folders in the Airoha IoT SDK.

**Table 2. Introduction of Important Folders in the Airoha IoT SDK**

| Relative Path                                                                                                                                                                                                                                                                                                                      | Meaning                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| mcu side:<br>airoha/ <b>mcu/modules</b> /hal/airoha/inc<br>airoha/ <b>mcu/modules</b> /hal/airoha/ab162x<br><br>risc-v side:<br>airoha/ <b>risc-v</b> /drivers/chip/ab162x<br>airoha/ <b>risc-v</b> /drivers/chip/inc<br><br>common part:<br>airoha/ <b>common</b> /drivers/chip/ab162x<br>airoha/ <b>common</b> /drivers/chip/inc | Series chip driver of HAL               |
| header file: zephyr/include/zephyr/drivers<br>source code: zephyr/drivers<br>...<br>airoha/mcu/drivers                                                                                                                                                                                                                             | Common API drivers<br>(header & source) |
| airoha/mcu/applications<br>airoha/risc-v/projects/applications                                                                                                                                                                                                                                                                     | Application folder                      |
| airoha/mcu/subsys/airoha<br>airoha/risc-v/middleware/airoha                                                                                                                                                                                                                                                                        | Middleware of airoha                    |
| airoha/mcu/boards/airoha                                                                                                                                                                                                                                                                                                           | Board configuration (HW related)        |
| airoha/mcu/dts/arm/airoha/ab162x<br>airoha/mcu/dts/bindings/ab162x                                                                                                                                                                                                                                                                 | Device tree related                     |
| airoha/mcu/prebuilt/hal<br>airoha/mcu/prebuilt/middleware<br>airoha/mcu/prebuilt/system                                                                                                                                                                                                                                            | Prebuilt libraries                      |

## Airoha IoT SDK for BT Data Build Environment Guide

### 4 Building a Project using the SDK

```

user@6eb3ee628df: /workdir
/workdir/airoha/risc-v/tools/toolchain/xpack-riscv-none-elf-gcc-13.2.0-2/bin/riscv-none-elf-objcopy -O binary --only-section=.log_str /workdir/out/ab1623m_evk/multi_thread/multi_thread/riscv/hello_world/feature/hello_world.elf /workdir/out/ab1623m_evk/multi_thread/multi_thread/riscv/hello_world/feature/risc_v_log_str.bin
[Airoha][RISC-V] generate disassembly file for debugging...
[Airoha][RISC-V] generate binary firmware...
[306/307] Building C object zephyr/CMakeFiles/zephyr_final.dir/_/_riscv/hello_world/feature/hello_world.c.o
[307/307] Linking CXX executable zephyr/zephyr.elf
Memory region      Used Size  Region Size  %age Used
FLASH:             142788 B    948 KB      14.71%
SRAM:              92432 B    384 KB      23.51%
SRAM_BT:           0 GB      16 MB       0.00%
ROM_LOG_STRING:    62337 B    32 MB       0.19%
IDT_LIST:          0 GB      2 KB        0.00%
Generating files from /workdir/out/ab1623m_evk/multi_thread/zephyr/zephyr.elf for board: ab1623m_evk
[13/19] Completed 'mcuboot'
[15/19] No install step for 'multi_thread'
[16/19] Completed 'multi_thread'
[17/19] [Airoha] collect firmware and change firmware name in flash_download.cfg
[Airoha][Collect Firmware] obtain zephyr version: 0x03070100
[Airoha][Collect Firmware] create directory /workdir/out/ab1623m_evk/multi_thread/download
[Airoha][Collect Firmware] create directory /workdir/out/ab1623m_evk/multi_thread/debug
[Airoha][Collect Firmware] create directory /workdir/out/ab1623m_evk/multi_thread/debug/multi_thread
[Airoha][Collect Firmware] copy debug files which file extension match ('.dts', '.elf', '.map', '.bin', '.hex', '.yaml', '.config', '.cmd', '.lst', '.json', '.symbols', '.stat')
from /workdir/out/ab1623m_evk/multi_thread/zephyr to /workdir/out/ab1623m_evk/multi_thread/debug/multi_thread
[Airoha][Collect Firmware] copy firmware file from /workdir/out/ab1623m_evk/multi_thread/multi_thread/zephyr/zephyr.signed.bin to /workdir/out/ab1623m_evk/multi_thread/download/
multi_thread.signed.bin
[Airoha][Collect Firmware] copy all other download binary files from /workdir/out/ab1623m_evk/multi_thread/multi_thread/download to /workdir/out/ab1623m_evk/multi_thread/download
d
[Airoha][Collect Firmware] copy debug files which file extension match ('.dis', '.bin', '.map', '.elf', '.c', '.ld') from /workdir/out/ab1623m_evk/multi_thread/multi_thread/risc
v/hello_world/feature to /workdir/out/ab1623m_evk/multi_thread/debug/riscv/hello_world/feature
[Airoha][Collect Firmware] create directory /workdir/out/ab1623m_evk/multi_thread/debug/mcuboot
[Airoha][Collect Firmware] copy debug files which file extension match ('.dts', '.elf', '.map', '.bin', '.hex', '.yaml', '.config', '.cmd', '.lst', '.json', '.symbols', '.stat')
from /workdir/out/ab1623m_evk/multi_thread/mcuboot/zephyr to /workdir/out/ab1623m_evk/multi_thread/debug/mcuboot
[Airoha][Collect Firmware] copy firmware file from /workdir/out/ab1623m_evk/multi_thread/mcuboot/zephyr/zephyr.bin to /workdir/out/ab1623m_evk/multi_thread/download/mcuboot.bin
[Airoha][Collect Firmware] build dts: C:/workdir/out/ab1623m_evk/multi_thread/multi_thread, /workdir/out/ab1623m_evk/multi_thread/mcuboot
[Airoha][Collect Firmware] default project: multi_thread
[Airoha][Collect Firmware] firmware of default project: multi_thread.signed.bin
[Airoha][Collect Firmware] bootloader project: mcuboot
[Airoha][Collect Firmware] firmware of bootloader project: mcuboot.bin
[Airoha][Collect Firmware] replace image-0.bin => multi_thread.signed.bin in flash_download.cfg
[Airoha][Collect Firmware] replace bootloader.bin => mcuboot.bin in flash_download.cfg
[Airoha][Collect Firmware] remove download entry of secure_hdr0.bin due to CONFIG_AIR_SECURITY_SECURE_BOOT_ENABLE=y not matched
[Airoha][Collect Firmware] remove download entry of secure_hdr1.bin due to CONFIG_AIR_SECURITY_SECURE_BOOT_ENABLE=y not matched
[Airoha][Collect Firmware] represent as hex format
[Airoha][Collect Firmware] generate /workdir/out/ab1623m_evk/multi_thread/download/flash_download.cfg
[Airoha][Collect Firmware] generate /workdir/out/ab1623m_evk/multi_thread/download/flash_download.cfg done!
[18/19] [Airoha] check sysbuild result, especially the address defined by the cross core variable
[Airoha][SysBuild Check] time: 0 min 0 seconds
[19/19] [Airoha][Build] print sysbuild time
[Airoha][Build] Real Build Time: 0 min 14 s
[Airoha][Build] Total Build Time: 0 min 15 s
Build Result: PASS
user@6eb3ee628df: /workdir$ ./build.py ab1623m_evk multi_thread

```

**Figure 7. Screenshot of Building the Multi\_Thread Sample Project**

As demonstrated by Figure 7, when entering the command line environment of the Docker image, developers can input the build command to initiate the construction of a simple example project.

The most basic build command syntax is `./build.py <BOARD> <PROJECT_BRIEF>`.

For the project build configuration, as shown in Figure 8, the actual build directive is `./build.py ab1623m_evk multi_thread`.

```

airoha > mcu > tools > mapping_proj > ab162x > ! project_list.yaml > ...
62  ab1623m_evk_multi_thread:
63      sysbuild_conf: config/ab1623m_evk/sysbuild_with_mcuboot.conf
64      proj: airoha/mcu/samples/templates/multi_thread
65      mcuboot:
66          # path relative to airoha/mcu/applications/mcuboot/boot/zephyr
67          DTC_OVERLAY_FILE: config/ab162x_evb/mcuboot.overlay
68          CONF_FILE: config/ab162x_evb/air_mcuboot.conf
69      multi_thread:
70          # path relative to airoha/mcu/templates/multi_thread
71          DTC_OVERLAY_FILE: config/ab1623m_evk/app.overlay
72          CONF_FILE: config/ab1623m_evk/multi_thread.conf

```

**Figure 8. Project Setting of Multi\_Thread for Board ab1623m\_evk**

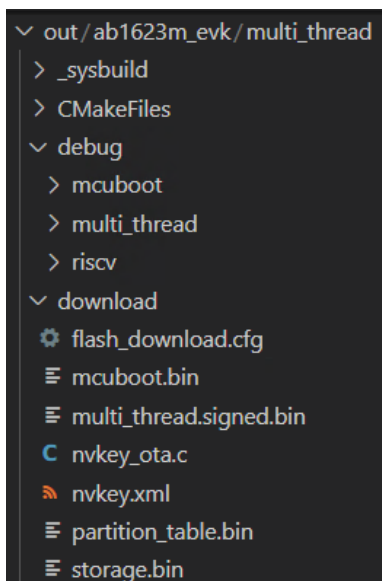
The following describes some output files generated after project completion. Figure 9 shows these files.

The path of the project's debug directory is `out/<BOARD>/<PROJECT_BRIEF>/debug`. It contains various files needed to analyze the issue, such as kconfig result, final linker script, device tree result, disassembly file, elf file, map file, etc.

## Airoha IoT SDK for BT Data

### Build Environment Guide

The path of the project's firmware directory is **out/<BOARD>/<PROJECT\_BERIEF>/download**. It contains all the firmware that can be downloaded.



**Figure 9. Build Result of Multi\_Thread for Board ab1623m\_evk**

Note that build.py is continuously evolving. Enter ./build.py in the command line to obtain the latest detailed instructions. The following are some of its examples and basic syntax.

**Table 3. Syntax and Examples of Using Build.py**

| Command                                            | Meaning                                                                                                                     |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| ./build.py ab1623m_evk multi_thread                | Build multi_thread project with ab1623m_evk board.                                                                          |
| ./build.py ab1623m_evk multi_thread rebuild        | Clean out folder of this project and then build multi_thread project with ab1623m_evk board.                                |
| ./build.py all                                     | Build all projects.                                                                                                         |
| ./build.py list   grep "^ab1623m_evk"              | List all projects with ab1623m_evk board.                                                                                   |
| ./build.py ab1623m_evk '*'                         | Build all projects with ab1623m_evk board.                                                                                  |
| ./build.py ab1623m_evk '(*.mouse *.dongle)'        | Build all projects with board ab1623m_evk and project names containing "mouse", or "dongle".                                |
| ./build.py 'ab162.*' '(?!.*(?:thread debug demo))' | Build all projects whose board name contains 'ab162' and project names that does not contain "thread", "debug", and "demo". |

## Airoha IoT SDK for BT Data Build Environment Guide

# 5 Creating and Configuring a Project

## 5.1 Creating a New Project Using an Existing Project

This section provides detailed information about creating your own project from an existing project. The following example shows how to create a new project named `example_project` on `ab1623m_evk` using `multi_thread` project as a reference.

The steps to create a new project using an existing project are very simple and are described below.

First, copy the entire `multi_thread` folder and rename it to `example_project`.

| Name            | Date modified   | Type        | Size |
|-----------------|-----------------|-------------|------|
| .git            | 9/29/2025 09:09 | File folder |      |
| multi_thread    | 9/11/2025 17:00 | File folder |      |
| single_thread   | 9/11/2025 17:00 | File folder |      |
| example_project | 9/29/2025 10:02 | File folder |      |

**Figure 10. Screenshot after Copying Multi\_Thread to Example\_Project**

The second step is to delete some unnecessary files and modify some words related to `multi_thread`. As mentioned above, according to the example requirements, only the relevant configuration of `ab1623m_evk` is required. Thus, it is necessary to delete the useless configurations of other development boards.

| samples > templates > example_project > config |                 |             |  |
|------------------------------------------------|-----------------|-------------|--|
| Name                                           | Date modified   | Type        |  |
| ab1615_fpga                                    | 9/29/2025 10:02 | File folder |  |
| ab1620_fpga                                    | 9/29/2025 10:02 | File folder |  |
| ab1623m_evb                                    | 9/29/2025 10:02 | File folder |  |
| ab1623m_evk                                    | 9/29/2025 10:02 | File folder |  |

**Figure 11. Screenshot of Board Configuration in Example\_Project.**

| samples > templates > example_project > config > |                 |             |  |
|--------------------------------------------------|-----------------|-------------|--|
| Name                                             | Date modified   | Type        |  |
| ab1623m_evk                                      | 9/29/2025 10:02 | File folder |  |

**Figure 12. Screenshot of Board Configuration in Example\_Project after Deleting Unnecessary Files**

Modify the term `multi_thread` to the new project name `example_project` by utilizing the editor's replace function.

Furthermore, in addition to replacing file contents containing `multi_thread`, developers also need to handle replacing file names containing `multi_thread`.

Finally, add the configuration of `example_project` to the `project_list.yaml` file according to the specified format.

## Airoha IoT SDK for BT Data Build Environment Guide

YAML (YAML Ain't Markup Language) is a human-readable data serialization format commonly used for configuration files and data exchange. It is designed to be simple, easy to read, and easy to write, while also supporting complex data structures. YAML's syntax is very intuitive, supports nested structures, and is more concise than JSON.

Due to these advantages, it is widely used in the Zephyr codebase. Airoha also uses it as the project configuration format.

```

airoha > mcu > tools > mapping_proj > ab162x > ! project_list.yaml > ...
75 | ab1623m_evk__example_project_brief:
76 |   ... sysbuild_conf: config/ab1623m_evk/sysbuild_with_mcuboot.conf
77 |   ... proj: airoha/mcu/samples/templates/example_project
78 |   ... mcuboot:
79 |     ... # path relative to airoha/mcu/applications/mcuboot/boot/zephyr
80 |     ... DTC_OVERLAY_FILE: config/ab162x_evb/mcuboot.overlay
81 |     ... CONF_FILE: config/ab162x_evb/air_mcuboot.conf
82 |   ... example_project:
83 |     ... # path relative to airoha/mcu/templates/example_project
84 |     ... DTC_OVERLAY_FILE: config/ab1623m_evk/app.overlay
85 |     ... CONF_FILE: config/ab1623m_evk/example_project.conf

```

**Figure 13. Project Settings Related to Example\_Project**

Developers should understand the following information from Figure 13:

- Line 75: The project uses ab1623m\_evk as the board and example\_project\_brief as the project brief.
- Line 76: The example\_project needs to be built together with mcuboot.
- Line 77: The path of the example\_project.
- Line 78~81: The configuration of the mcuboot subproject.
- Line 82~85: The configuration of the main.



Note: Ensure consistency between the **main project name on line 82** and the **last-level directory name on line 77**.

## 5.2 Configuring a Project

Project configuration involves two aspects: specifying the device tree overlay file by the DTC\_OVERLAY\_FILE parameter and specifying the Kconfig file by the CONF\_FILE parameter.

Zephyr uses devicetree to describe hardware resources (such as GPIO, I2C, SPI) and their configurations. The devicetree overlay files, typically suffixed with .overlay, and allow customization of the devicetree based on specific application requirements. For a comparison of devicetree and Kconfig, refer to [Devicetree versus Kconfig](#) for detailed information. Zephyr's description of Kconfig is very detailed, and developers can also refer to [Configuration System \(Kconfig\)](#) for the most authoritative information.

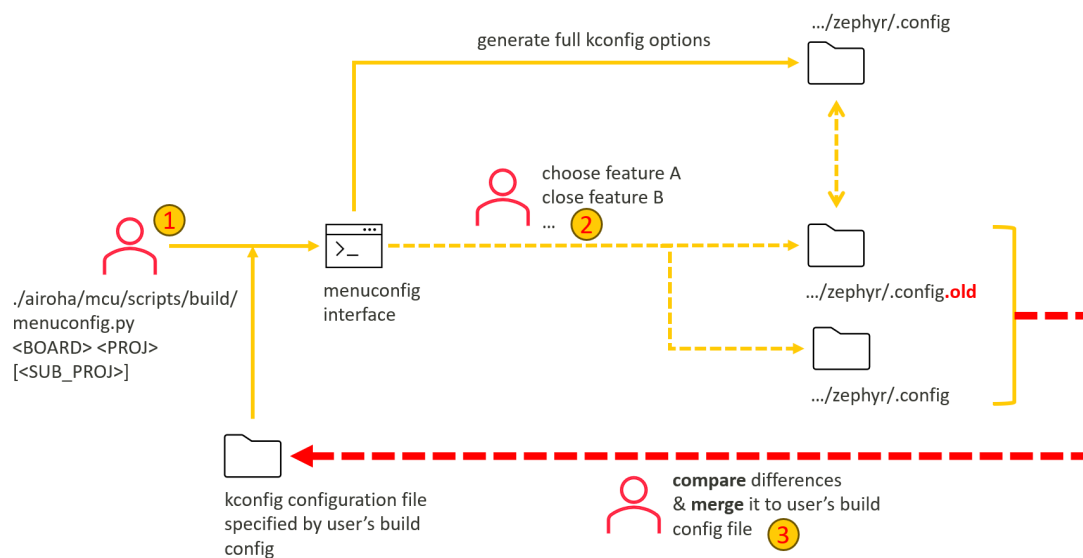
This section introduces menuconfig, a curses-based interface that operates in the terminal. It assists developers in exploring the available Kconfig options and making **temporary** changes.

## Airoha IoT SDK for BT Data Build Environment Guide

The following are some tips for using menuconfig:

- Navigate the menu with the **arrow** keys.
- Use **Space** and **Enter** to enter menus and toggle values. Menus appear with ---> next to them. Press **ESC** to return to the parent menu.
- **Boolean** configuration options are shown with [ ] brackets, while numeric and **string-valued configuration** symbols are shown with ( ) brackets.
- Press **?** to display information about the currently selected symbol, including its help text.
- Press **Q** in the menuconfig interface to bring up the save-and-quit dialog (if there are changes to save).

After understanding the basic operations of menuconfig, the next step is to understand the general process of configuring a project's feature option, as shown in Figure 14.



**Figure 14. Overall Flow of Configuring Kconfig Options of a Project**

First, developers can enter commands to run menuconfig.py, as shown in Figure 14. Figure 15 shows the results.

## Airoha IoT SDK for BT Data Build Environment Guide

```
(Top)
Zephyr Kernel Configuration

() customised initial configuration file
[*] cross memory buffer for multiple cores accesses
[ ] Custom Bluetooth assert implementation
[ ] enable boot time measure, include MCUBoot and Main application
[ ] wrap printk function to add newlines if needed
[*] Use Airoha's memcpy/memset function
[*] wrap malloc/calloc/realloc/free... C lib functions to zephyr kernel apis
[*] wrap k_malloc/k_calloc/k_realloc/k_aligned_alloc/k_free... zephyr kernel function, add some debug information
[*] enable wrap __stack_chk_fail so that we can trigger exception dump when stack check fail because of stack corruption
   Compiler mechanism for stack protection (-fstack-protector-strong) --->
[*] generate flash_download.cfg for airoha's download tool
[*] Vector Table at header of firmware, n means reset_handler at header of firmware
[*] allow project level linker script snippets to be used in SoC common linker script templates
(0) limit the amount of RAM remaining to at least this value
(0) limit the amount of flash memory remaining to at least this value
(393216) limit the amount of retention RAM to ensure low power
   Booting Method (booting from flash) --->
   Airoha Daemon Work Queue Options --->
[*] Airoha Board Support Package level flash api
[ ] Airoha Block Data Management
   Devicetree Info ----
   Modules --->
[*] Airoha ab1623m_evk
   Board Options ----
   Hardware Configuration --->
   ARM Options --->
   General Architecture Options --->
[*] MPU features --->
[*] Assign appropriate permissions to kernel areas in SRAM
[ ] Support code/data section relocation
   DSP Options ----
   Floating Point Options --->
   Cache Options --->
[ ] Custom arch_cpu_idle implementation
   General Kernel Options --->
   Device Options --->
   Initialization Priorities --->
↓↓↓↓↓↓↓↓↓↓↓↓↓↓
[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[0] Load                  [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

**Figure 15. Screenshot of the Results after Executing Menuconfig.py**

Upon accessing the menuconfig page, the previously mentioned techniques can be utilized to modify Kconfig options. The following example demonstrates how to change the stack size of the Airoha daemon work queue thread.

```
Jump to symbol/choice/menu/comment
daemon
AIR_DAEMON_WORKQUEUE_ENABLE(=y) "Airoha daemon workqueue"
AIR_DAEMON_WORKQUEUE_STACK_SIZE(=1024) "Airoha daemon workqueue stack size"
DAEMON_WORK_Q_INIT_PRIORITY(=0)
menu "Airoha Daemon Work Queue Options"
```

**Figure 16. Screenshot of the Results after Entering '/', Entering 'daemon' and Pressing the Down Arrow**

Next, press the **Enter** key twice to access the window for adjusting the daemon workqueue stack size.

# Airoha IoT SDK for BT Data

## Build Environment Guide

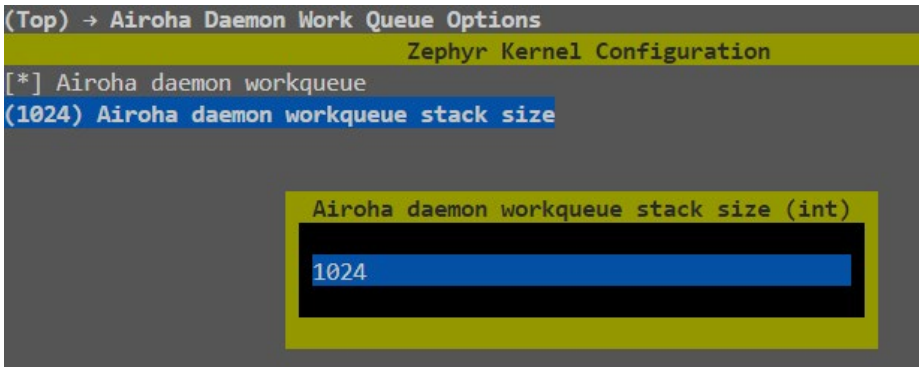


Figure 17. Kconfig Option Adjustment Window

Developers can then use the backspace key to delete the existing settings, enter the desired settings, exit the menuconfig interface, and save the changed settings.

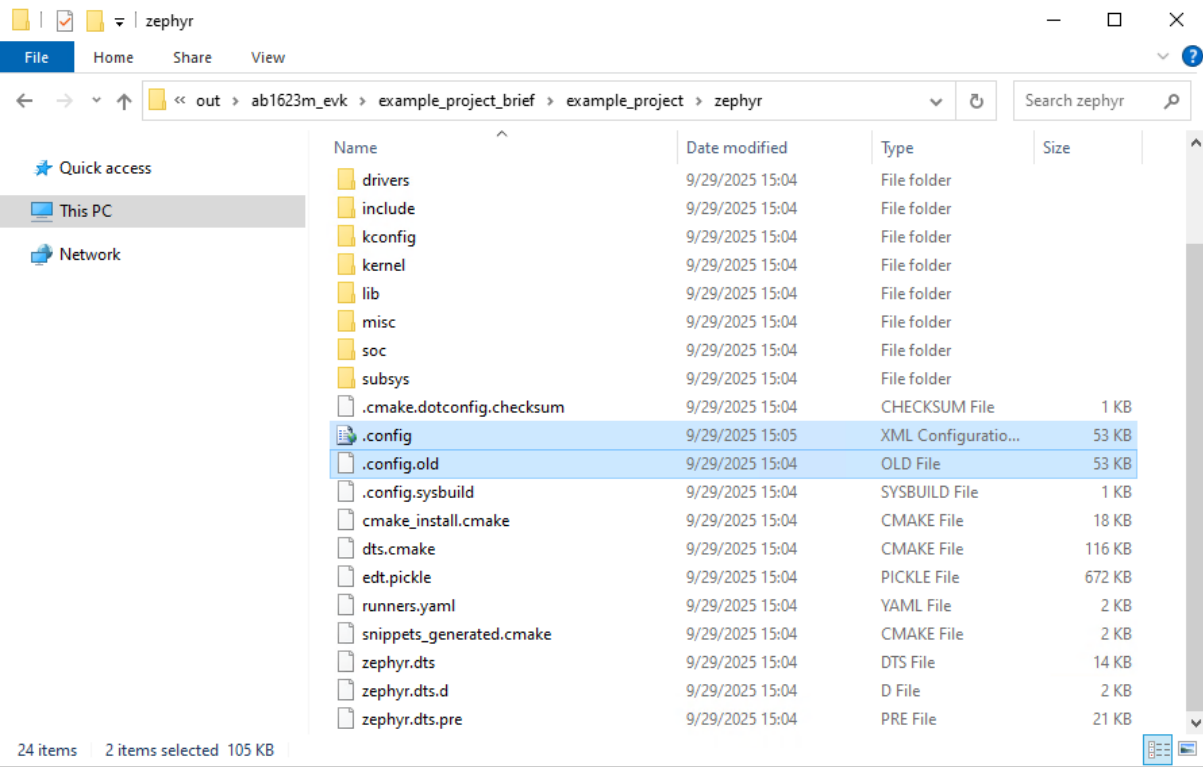


Figure 18. The Result Files after Changing the Kconfig Options

In the final step, developers can use comparison tools to compare the changes between .config and .config.old, and write the expected differential Kconfig options into the project configuration CONF\_FILE file to make the changes to the Kconfig permanent.

## Airoha IoT SDK for BT Data Build Environment Guide

```

! project_list.yaml M  example_project.conf  .config  .config ↔ .config.old ×
out > ab1623m_evk > example_project_brief > example_project > zephyr > .config.old
161 # 161 #
162 # Airoha Daemon Work Queue Options 162 # Airoha Daemon Work Queue Options
163 # 163 #
164 CONFIG_AIR_DAEMON_WORKQUEUE_ENABLE=y 164 CONFIG_AIR_DAEMON_WORKQUEUE_ENABLE=y
165- CONFIG_AIR_DAEMON_WORKQUEUE_STACK_SIZE=2048 165+ CONFIG_AIR_DAEMON_WORKQUEUE_STACK_SIZE=1024
166 # end of Airoha Daemon Work Queue Options 166 # end of Airoha Daemon Work Queue Options
167 167

```

**Figure 19. Difference between .config and .config.old**

```

! project_list.yaml M  example_project.conf ×
airoha > mcu > samples > templates > example_project > config > ab1623m_evk > example_project.conf
148
149 # Merge the temporarily modified Kconfig option into .conf file
150 # to ensure the change is permanent for this project.
151 CONFIG_AIR_DAEMON_WORKQUEUE_STACK_SIZE=2048
152
153

```

**Figure 20. Merge Temporarily Changes to Configuration File**

## Airoha IoT SDK for BT Data Build Environment Guide

### Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to Airoha Technology Corp. and/or its affiliates (collectively “Airoha”) or its licensors and is provided solely for Your internal use with Airoha’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against Airoha or any of Airoha’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify Airoha for any loss or damages suffered by Airoha for Your unauthorized use or disclosure of this Document, in whole or in part.

Airoha and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. Airoha does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY AIROHA IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. AIROHA SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. AIROHA SHALL NOT BE RESPONSIBLE FOR ANY AIROHA DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, Airoha makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does Airoha assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating Airoha’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.